

Mixing FM-indexes and CSAs: backward search over an order-1 rank encoding

Travis Gagie

Center for Biotechnology and Bioengineering (CeBiB), Chile

Preliminary version, September 30, 2026

Abstract

FM-indexes and compressed suffix arrays (CSAs) are often treated as interchangeable, but they behave differently as the alphabet grows. An FM-index step costs about one cache miss per level of a wavelet tree, so it gets slower with the alphabet size. A CSA step is a binary search whose range shrinks as characters get rarer. We describe a simple hybrid. Each character of the text is replaced by the rank of its frequency among the characters that follow the previous character. We backward-search on this encoding, which is over a small, skewed alphabet, and recover the one piece of information the encoding loses (the first character of the pattern) with a single CSA-like step on an array we call Ψ_E . Counting is exact, and locating works with standard suffix-array sampling. A prototype on synthetic repetitive data shows that the hybrid is the fastest of the indexes we tried at intermediate alphabet sizes with 1% noise, but even its compact version is 1.7 to 3.9 times larger than a compressed run-length CSA or FM-index of the original text, because the encoding and Ψ_E together have more runs than the original Burrows–Wheeler transform. Whether that changes on real data, such as parses and minimizer digests, is the main open question.

1 Introduction

Indexes for highly repetitive text collections, such as pangenomes, are usually built for small alphabets: DNA has four characters. Increasingly, though, we want to index repetitive sequences over large alphabets. In two-level indexing we index the parse of a text, whose alphabet is the dictionary of distinct phrases [3, 14]. Minimizer digests of pangenomes treat each minimizer as a meta-character [1]. Collections of versioned documents can be viewed as sequences of words [6]. These sequences are still repetitive, so their Burrows–Wheeler transforms (BWTs) still have relatively few runs, but the alphabets have thousands or millions of symbols.

For such data, the choice between the two classic families of compressed indexes matters. The FM-index [8] represents the BWT with a wavelet tree, and one backward-search step costs one rank query per level of the tree: about $\lg \sigma$ levels for an alphabet of size σ , or about H_0 levels with a Huffman-shaped tree. Each level is a separate bitvector, so each level can cost a cache miss. A compressed suffix array (CSA) [13] instead stores the permutation Ψ , and one step is a binary search in the part of Ψ belonging to one character, whose length is that character’s frequency (or, in a run-length compressed CSA, its number of runs [3]). For large alphabets those parts are short, so CSAs tend to win; for small alphabets FM-indexes tend to win.

The idea, informally. In most real sequences, the previous character tells us a lot about the next one. Instead of storing each character, we store how *surprising* it is: its rank by frequency

among the characters that follow the previous character in the text. In English, after “q” comes “u” almost always, so “u” after “q” becomes a 1. The encoded text E is then mostly small numbers (on our test data, mostly 1s), so backward search on E can use a very shallow wavelet tree. We can encode a pattern P the same way, using the text’s rankings, and search for it in E . The catch is that the encoding of P only makes sense given P ’s first character: the string of ranks “1, 1, 1” means “ANA” after a B but “BAN” after a $_$. So after the search, we must keep only the encoded suffixes that are preceded in the text by $P[0]$. We do that with one lookup in an array Ψ_E of the (unencoded) characters that precede the encoded suffixes, stored the way a CSA stores Ψ . The result is an FM-index for all but one step of the search, and a CSA for the last one, which is the only step that has to deal with the large alphabet.

What we found. The hybrid is correct and simple to implement. On synthetic repetitive data it has the following properties:

- Its query time barely depends on the alphabet size, because the Huffman-shaped wavelet tree over the encoding has depth about 2 whatever the alphabet.
- At intermediate alphabet sizes (10 to 10,000) with 1% noise, it is the fastest of the indexes we tried, by 1.1 to 1.8 times (the smallest margin is within measurement noise).
- It is never the smallest. The encoding and Ψ_E together have 1.6 to 4.1 times as many runs as the BWT of the text, so even the compact hybrid is 1.7 to 3.9 times larger than a compressed run-length CSA of the text, and no faster than it.

So on this data the hybrid buys speed with space, and its compact version is dominated. The question that decides whether it is useful is how many runs the encoding and Ψ_E have on real parses and digests.

Organization. Section 2 reviews FM-indexes, CSAs and their run-length versions on a small example, with attention to memory locality. Section 3 describes the hybrid on the same example, proves that counting is exact, and sketches locating. Section 4 discusses related work, Section 5 our implementation, Section 6 experiments, and Section 7 open questions, including what the hybrid means for move structures.

2 Background

Let $T[0..n-1]$ be a text over an alphabet of size σ , terminated by a unique smallest character $\$$. The suffix array $SA[0..n-1]$ lists the starting positions of the suffixes of T in lexicographic order. The BWT $L[0..n-1]$ has $L[i] = T[SA[i]-1]$ (cyclically), and $F[i] = T[SA[i]]$ is the first character of the i th suffix. We write r for the number of runs of equal characters in L , and n_c and r_c for the number of occurrences of c in T and the number of runs of c in L .

Our running example is

$$T = \text{BANANA_BANDANA_BANANA_CABANA\$},$$

with $n = 29$ and $\sigma = 7$ (ordered $\$ < A < B < C < D < N < _$). Figure 1 shows its sorted suffixes. The BWT has $r = 13$ runs.

i	$SA[i]$	$\Psi[i]$	suffix $T[SA[i]..n-1]$ (first character is $F[i]$)	$L[i]$
0	28	14	\$	A
1	27	0	A\$	N
2	23	13	ABANA\$	C
3	25	19	ANA\$	B
4	1	20	ANANA_BANDANA_BANANA_CABANA\$	B
5	16	21	ANANA_CABANA\$	B
6	11	22	ANA_BANANA_CABANA\$	D
7	3	23	ANA_BANDANA_BANANA_CABANA\$	N
8	18	24	ANA_CABANA\$	N
9	8	25	ANDANA_BANANA_CABANA\$	B
10	13	26	A_BANANA_CABANA\$	N
11	5	27	A_BANDANA_BANANA_CABANA\$	N
12	20	28	A_CABANA\$	N
13	24	3	BANA\$	A
14	0	4	BANANA_BANDANA_BANANA_CABANA\$	\$
15	15	5	BANANA_CABANA\$	-
16	7	9	BANDANA_BANANA_CABANA\$	-
17	22	2	CABANA\$	-
18	10	6	DANA_BANANA_CABANA\$	N
19	26	1	NA\$	A
20	2	7	NANA_BANDANA_BANANA_CABANA\$	A
21	17	8	NANA_CABANA\$	A
22	12	10	NA_BANANA_CABANA\$	A
23	4	11	NA_BANDANA_BANANA_CABANA\$	A
24	19	12	NA_CABANA\$	A
25	9	18	NDANA_BANANA_CABANA\$	A
26	14	15	_BANANA_CABANA\$	A
27	6	16	_BANDANA_BANANA_CABANA\$	A
28	21	17	_CABANA\$	A

Figure 1: Sorted suffixes of $T = \text{BANANA_BANDANA_BANANA_CABANA\$}$. Horizontal rules separate the blocks of suffixes starting with each character. Blue rows are the suffixes starting with **ANA**; orange rows are those starting with **BANA**. Both an FM-index and a CSA find the orange interval from the blue one in one step.

2.1 Backward search and FM-indexes

Backward search finds the SA interval of each suffix of a pattern P , from shortest to longest. If $[s, e]$ is the interval for $P[j + 1..m - 1]$ and $c = P[j]$, then the interval for $cP[j + 1..m - 1]$ is

$$[C[c] + \text{rank}_c(L, s), C[c] + \text{rank}_c(L, e + 1) - 1],$$

where $C[c]$ is the number of characters in T smaller than c and $\text{rank}_c(L, i)$ counts the copies of c in $L[0..i - 1]$. In Figure 1, the interval for **ANA** is $[3, 8]$, $L[3..8] = \text{BBBDNN}$ contains three Bs, and $C[\text{B}] = 13$, so the interval for **BANA** is $[13, 15]$.

The FM-index [8] answers rank_c queries with a *wavelet tree* [12] over L . The root holds one bit per character of L saying whether it is in the first or second half of the alphabet; each child holds the subsequence of characters on its side, and so on (Figure 2a). A rank query descends from the root to c 's leaf, doing one bitvector rank per level. A balanced tree has $\lceil \lg \sigma \rceil$ levels. A Huffman-shaped tree [16] has fewer than $H_0(T) + 1$ levels on average, where H_0 is the empirical entropy.

Locality. The bitvectors of different levels are stored in different places in memory, so each level of a descent is essentially a random access. For DNA this means two or three cache misses per step. For an alphabet of 10^5 symbols it means about 17, and the FM-index becomes slow.

2.2 Compressed suffix arrays

A CSA [13, 21] stores $\Psi[i] = SA^{-1}[SA[i] + 1]$: the row of the suffix that starts one position later. Equivalently, $\Psi[i]$ is the position in L of the character $F[i]$. Within the block of rows whose suffixes start with a given character c , Ψ is increasing, because removing the same first character from each of those suffixes keeps them in the same relative order. To extend the interval $[s, e]$ of $P[j + 1..m - 1]$ by c , we binary-search c 's block of Ψ for the values that lie in $[s, e]$. In the example, the B block of Ψ is rows 13..16 with values 3, 4, 5, 9. The values in $[3, 8]$ are in rows 13..15, so again the interval for **BANA** is $[13, 15]$ (Figure 2b).

Locality. The binary search runs over a contiguous range of length n_c . Its cost depends on how often c occurs, not on how many other characters there are. For a large alphabet most characters are rare, their ranges are short, and a step costs a few probes into one small region of memory.

2.3 Run-length versions

When T is repetitive, r is much smaller than n , and both families have run-length versions whose size is proportional to r rather than n .

- The **RLFM-index** [16] stores a sparse bitvector marking where the runs of L start, a wavelet tree over the r run heads, and, for each character, where its runs start in F . A rank query finds the run containing the position, does a rank on the run heads, and adds an offset. In the example the run heads are **ANCBDBNA\$_NA**, with run lengths 1, 1, 1, 3, 1, 2, 1, 3, 1, 1, 3, 1, 10.
- The **RLCSA** [17, 23] stores Ψ at the level of runs. Brown, Gagie, Manzini, Navarro and Sciortino [3] describe it as a permutation Ψ' on the runs, which is again increasing within each character's block, together with sparse bitvectors B_L and B_F marking where runs start in L and F . A step is a binary search over the r_c runs of c . With interpolative coding it takes $O(r \log(n/r) + r \log \sigma + \sigma)$ bits and $O(\log r_c)$ time per step.

(a) **FM-index**: a balanced wavelet tree over L

depth	split (0 1)	subsequence of L	bits
0	\$ABC DN_	ANCBBDNBNBNNNA\$__NAAAAAAAAA	01000011101110011110000000000
1	\$A BC	ACBBBBBAAAAAAAAA	0111110000000000000
2	\$ A	AA\$AAAAAAAAA	11011111111111
2	B C	CBBBB	10000
1	DN _	NDNNNNN__N	00000001110
2	D N	NDNNNNNN	10111111

Each node splits its alphabet into a left half (bit 0) and a right half (bit 1). Computing $\text{rank}_B(L, i)$ descends three levels, $\$ABC DN_ \rightarrow \$ABC \rightarrow BC$: one bitvector rank, and potentially one cache miss, per level.

(b) **CSA**: the increasing blocks of Ψ

c	rows	Ψ values
\$	[0..0]	14
A	[1..12]	0, 13, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28
B	[13..16]	3, 4, 5, 9
C	[17..17]	2
D	[18..18]	6
N	[19..25]	1, 7, 8, 10, 11, 12, 18
_	[26..28]	15, 16, 17

To prepend B to ANA (rows [3, 8]), binary-search the B block for values in [3, 8]. They are 3, 4, 5, in rows 13..15: one binary search in one short, contiguous list.

Figure 2: The two ways of taking the backward step from ANA to BANA in Figure 1.

The locality argument carries over. An RLFM-index step costs a wavelet-tree descent over the run heads, about $\lg \sigma$ levels. An RLCSA step costs a binary search over r_c runs, and for a large alphabet r_c is usually small. Brown et al. [3] point out that Ordóñez, Navarro and Brisaboa [20] confirmed experimentally that RLCSAs do well on large alphabets with few runs per character. Our experiments (Section 6) agree, and also show FM-type indexes doing well on small alphabets.

3 The hybrid

3.1 The encoding

For each character a , list the distinct characters that follow occurrences of a in T (ignoring the final $\$$), ordered by how often they follow a , with ties broken consistently (we use the alphabet order). Call the position of b in a 's list its *rank after a* , $\rho_a(b)$, starting from 1. The *encoding* of T is

$$E[i] = \rho_{T[i-1]}(T[i]) \quad \text{for } 1 \leq i \leq n-2, \quad E[n-1] = \$.$$

We do not encode $T[0]$. Given $T[0]$ and the lists, T can be decoded from left to right. The lists hold one entry per distinct bigram in T .

Figure 3 shows the lists and the encoding for our example. After **A** comes **N** seven times, **_** three times and **B** once, so after an **A**, the ranks 1, 2 and 3 mean **N**, **_** and **B**. Most characters in this text are the most likely successor of their predecessor, and 21 of the 27 ranks in E are 1s.

A pattern $P[0..m-1]$ (without $\$$) is encoded the same way, using the text's lists:

$$Q[j] = \rho_{P[j-1]}(P[j]) \quad \text{for } 1 \leq j \leq m-1.$$

If some $P[j]$ never follows $P[j-1]$ in T , then P does not occur and we can stop immediately. We call $Q[1..m-1]$ the encoding of P and write $\text{enc}(P)$ for it. For example, $\text{enc}(\text{BANA}) = 1111$: **A** is the first choice after **B**, **N** after **A**, and **A** after **N**.

3.2 The array Ψ_E and counting

Let SA_E be the suffix array of $E[1..n-1]$, whose entries are positions $1..n-1$ of T . For each position i , the suffix $E[i..n-1]$ encodes $T[i..n-1]$ *given* the preceding character $T[i-1]$. We define

$$\Psi_E[k] = T[SA_E[k] - 1],$$

the unencoded character that precedes the k th encoded suffix. Ψ_E is to the E-order what L is to the ordinary order: it holds the preceding characters. But it holds the characters of T , not of E .

Lemma 1. *Let $P[0..m-1]$ be a pattern with $m \geq 2$ whose encoding $Q = \text{enc}(P)$ is defined, and let $0 \leq p \leq n-m-1$. Then $T[p..p+m-1] = P$ if and only if $T[p] = P[0]$ and $E[p+1..p+m-1] = Q[1..m-1]$.*

Proof. If $T[p..p+m-1] = P$, then for each $j \geq 1$, $E[p+j] = \rho_{T[p+j-1]}(T[p+j]) = \rho_{P[j-1]}(P[j]) = Q[j]$. Conversely, suppose $T[p] = P[0]$ and $E[p+1..p+m-1] = Q[1..m-1]$. Assume $T[p+j-1] = P[j-1]$ for some $j \geq 1$. Then $\rho_{P[j-1]}(T[p+j]) = E[p+j] = Q[j] = \rho_{P[j-1]}(P[j])$. Distinct successors of $P[j-1]$ have distinct ranks, so $T[p+j] = P[j]$. The claim follows by induction on j . $\square$

By the lemma, counting is exact:

1. Compute $Q = \text{enc}(P)$ from the rank lists.
2. Backward-search $Q[1..m - 1]$ in an FM-type index of E , giving an interval $[s, e]$ of SA_E .
3. Return $\text{rank}_{P[0]}(\Psi_E, e + 1) - \text{rank}_{P[0]}(\Psi_E, s)$.

If $m = 1$, the interval is the whole array and the answer is $n_{P[0]}$. No special case is needed at the start of the text: an occurrence at $T[0]$ appears as the suffix $E[1..n - 1]$ with $\Psi_E = T[0]$.

Example. To count **BANA**, we search for $Q = 111$. Its interval in SA_E is $[3, 12]$ (Figure 3): ten encoded suffixes start with 111 . They are preceded in T by the characters $\Psi_E[3..12] = \text{B_BBAAD_NN}$, and three of those are Bs, so **BANA** occurs three times. The other seven encode **BAN** after a **_**, **NAN** after an **A**, and **ANA** after a **D** or an **N**: once encoded, they look the same as **ANA** after a **B**.

3.3 Why this is a hybrid

All but one step of the search are FM-index steps on $\text{BWT}(E)$, whose alphabet is small and skewed. Its size is the largest number of distinct successors of any character, and in practice most entries are 1s and 2s, so a Huffman-shaped wavelet tree over $\text{BWT}(E)$ has depth about 2 (Section 6). The last step is a CSA step. If we store, for each character c , the sorted list of positions (or runs) of c in Ψ_E , then $\text{rank}_c(\Psi_E, \cdot)$ is a binary search in c 's list. That is exactly the operation on c 's block of Ψ in a CSA, and it benefits from a large alphabet in the same way. Concatenating these lists in character order lists the suffixes of T sorted by the key $(T[i - 1], E[i..n - 1])$. So the lists form a Ψ -like map from that order to the E-order, just as Ψ maps from F to L .

Computing $\text{enc}(P)$ needs one lookup per character in the rank lists. The lookups can be done for the whole pattern before the search starts, with good locality, since consecutive lookups are for consecutive characters of P .

3.4 Locating

Locating with suffix-array samples works unchanged. LF-steps on $\text{BWT}(E)$ move from the suffix of E at position i to the one at $i - 1$, and positions in E are positions in T . So we sample SA_E and walk to the nearest sample. The rows we need to locate are the positions k of $P[0]$ in $\Psi_E[s..e]$, which the CSA-style lists give directly, and the occurrence of P for row k starts at $SA_E[k] - 1$.

An r-index-style [9] locate with ϕ also seems possible, though we have not implemented it or checked it carefully. Sort the suffixes of T by the key $(T[i], E[i + 1..n - 1])$ and call this order SA' . The occurrences of P form one contiguous range of SA' . Consider the map from i to $i - 1$ in this order. Within the block of suffixes preceded by a character c , the suffixes are sorted by $(\rho_c(T[i]), E[i + 1..n - 1])$. So the map preserves relative order among suffixes with the same $T[i]$ and the same $T[i - 1]$. Two suffixes adjacent in SA' with the same first character and the same preceding character therefore stay adjacent, which is the property behind the toehold lemma and ϕ . Samples would be needed:

- at run boundaries of the preceding characters listed in SA' order;
- at the σ boundaries in SA' where the first character changes;
- at run ends of Ψ_E , to get an initial toehold from an r-index on E .

That is $O(r(E) + r(\Psi_E) + r' + \sigma)$ samples in total, where r' is the number of runs in that listing. SA' is a special case of the *local orderings* of Giancarlo et al. [10] (Section 4), so their framework may already give this.

(a) successors in rank order
(counts)

a	successors of a
A	N (7), _ (3), B (1)
B	A (4)
C	A (1)
D	A (1)
N	A (6), D (1)
_	B (2), C (1)

(b) the text and its encoding

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
$T[i]$	B	A	N	A	N	A	_	B	A	N	D	A	N	A	_
$E[i]$	-	1	1	1	1	1	2	1	1	1	2	1	1	1	2

i	15	16	17	18	19	20	21	22	23	24	25	26	27	28
$T[i]$	B	A	N	A	N	A	_	C	A	B	A	N	A	\$
$E[i]$	1	1	1	1	1	1	2	2	1	3	1	1	1	\$

(c) sorted encoded suffixes

k	$SA_E[k]$	$E[SA_E[k]..n-1]$	BWT(E)	$\Psi_E[k]$
0	28	\$	1	A
1	27	1\$	1	N
2	26	11\$	1	A
3	25	111\$	3	B
4	15	1111112213111\$	2	-
5	1	11111211112111112213111\$	\$	B
6	16	111112213111\$	1	B
7	2	11112111211121111112213111\$	1	A
8	17	11112213111\$	1	A
9	11	11121111112213111\$	2	D
10	7	111211121111112213111\$	2	-
11	3	11121112111211111112213111\$	1	N
12	18	1112213111\$	1	N
13	12	1121111112213111\$	1	A
14	8	112111211111112213111\$	1	B
15	4	112111211121111112213111\$	1	A
16	19	112213111\$	1	A
17	13	121111112213111\$	1	N
18	9	1211121111112213111\$	1	A
19	5	121112111211111112213111\$	1	N
20	20	12213111\$	1	N
21	23	13111\$	2	C
22	14	21111112213111\$	1	A
23	10	211121111112213111\$	1	N
24	6	2111211121111112213111\$	1	A
25	22	213111\$	2	-
26	21	2213111\$	1	A
27	24	3111\$	1	A

Figure 3: The hybrid for $T = \text{BANANA_BANDANA_BANANA_CABANA\$}$. (a) The rank lists. (b) The encoding $E[1..n-1]$. (c) The encoded suffixes in sorted order, with BWT(E) and Ψ_E . Rows with a colour are the interval of $111 = \text{enc}(\text{BANA})$; the orange ones have $\Psi_E = \text{B}$ and are the three occurrences of BANA. BWT(E) has 11 runs, against 13 for BWT(T), but Ψ_E has 22.

3.5 Space

The index consists of the rank lists (one entry per distinct bigram), a run-length representation of $\text{BWT}(E)$, and a run-length representation of Ψ_E . E is a local function of T (each $E[i]$ depends only on $T[i - 1..i]$), so a phrase of T 's LZ77 parse is also a copy in E except possibly for its first character, whose rank depends on the character before the phrase. Hence $z(E) \leq 2z(T)$, and $r(E) = O(z(T) \log^2 n)$ by Kempa and Kociumaka's bound [15]. But we see no reason for $r(E) \leq r(T)$ in general. Sorting by the encoding interleaves suffixes that differ in T but have the same encoding, which can split runs. And a point mutation changes two characters of E instead of one. Ψ_E is the bigger concern. It is a sequence over the full alphabet, and on our test data with $\sigma \geq 10$ it has 1.3 to 2.3 times as many runs as $\text{BWT}(T)$. The example is typical in this respect: $\text{BWT}(E)$ has fewer runs than $\text{BWT}(T)$ (11 against 13), but Ψ_E has 22.

4 Related work

We are not aware of this exact construction. The following are the closest ideas we know.

Context-dependent orderings. Giancarlo, Manzini, Restivo, Rosone and Sciortino [10] define BWT-like transforms in which the alphabet order used for sorting depends on the preceding context. They show that these *local orderings* can be inverted and searched in linear time and can be turned into r-indexes. The order SA' in Section 3.4 is a local ordering with a one-character context, and the frequency ranking is one particular choice of orderings. The E-order itself also permutes the first character by the preceding one, so it is a close relative rather than an instance. Choosing a global alphabet order to improve compression or reduce runs goes back at least to Chapin and Tate [4]; minimizing runs this way is NP-hard [2].

Edge ranks in the GBWT. Sirén et al.'s GBWT [22] stores, for each node of a pangenome graph, the BWT restricted to that node as runs of *edge ranks*: the position of the next node among the node's outgoing edges. That is our encoding (with ranks in node-ID order rather than by frequency), and it is used for the same reason: a huge alphabet of node IDs with small local out-degrees. The GBWT sorts by the real paths, however, not by their encoding.

Large-alphabet self-indexes. Word-based self-indexes [6] target natural-language text with a large alphabet of words. Two-level indexes [14, 3] index a text together with its parse; Deng et al. [5] index a parse without an index for the text itself. Brown et al. [3] note that RLCSAs may be more suitable than RLFM-indexes for the parse, which is part of the motivation for this work. The encoding itself is an order-1 symbol-ranking transform, in the spirit of Fenwick's symbol-ranking compressors [7].

A natural baseline suggested by the GBWT is an FM-index of T whose BWT is split into blocks by the character in F , with each block relabeled by rank among that character's predecessors. Every step would then be a rank over a small local alphabet, and relabeling within a block does not split runs. We have not implemented this baseline, and it may be the strongest competitor for the hybrid.

5 Implementation

Our prototype, in C++, reads a file of fixed-width integers, maps them to dense identifiers, computes the rank lists, E , $\text{BWT}(E)$ and Ψ_E , and builds suffix arrays with libsa [11]. It currently

only counts. The code is available at <https://github.com/TravisGagie/Hybrid-FM-CSA>. We implemented four interchangeable rank structures and used each both for $\text{BWT}(E)$ and for the BWT of T , to compare against standard indexes of T built with the same code.

Explicit run lists. For each character, the sorted list of its runs as pairs (start position, number of earlier copies), at 64 bits per run. A small jump table on the high bits of the position narrows the binary search. Applied to $\text{BWT}(T)$, this is an uncompressed RLCSA: the pairs are the entries of B_L and B_F for each run, stored explicitly. Searching on positions instead of run indices means no rank query on B_L is needed.

Elias–Fano run lists. A compressed RLCSA following Brown et al. [3], simplified. The runs, grouped by character, are stored as two Elias–Fano sequences: $c \cdot n + \text{start}$ (one predecessor query both finds c ’s block and searches it) and the run’s position in F (the paper’s B_F). The space is about $r(\lg(\sigma n/r) + 2) + r(\lg(n/r) + 2)$ bits, matching the paper’s bound, and it avoids the rank on B_L that the paper removes with run splitting and an extra bitvector.

Huffman-shaped wavelet tree. Over the whole sequence, as in an FM-index. Backward search only needs rank_c , never comparisons between characters, so a Huffman shape is optimal and Hu–Tucker is not needed.

Run-length Huffman-shaped wavelet tree. An RLFM-index: an Elias–Fano sequence of run starts, a Huffman-shaped wavelet tree over the run heads, and an Elias–Fano sequence of the runs’ positions in F .

In the hybrid, Ψ_E is always stored with Elias–Fano run lists, since it is used for one step per query and is the natural place for the CSA structure. The rank lists are stored as sorted arrays of (successor, rank) pairs per character.

The RLFM-index implementation is not tuned. Each rank query does an Elias–Fano predecessor search, a wavelet-tree descent at two adjacent positions, and two Elias–Fano accesses, and the two ends of a backward-search interval are handled separately. A careful implementation, such as the original layout of Mäkinen and Navarro [16] with a fast sparse-bitvector rank [19], would likely be faster, so we give its query times less weight than the others.

6 Experiments

Data. We generated synthetic repetitive sequences of 50 million 32-bit integers. A base sequence of 10^6 symbols is drawn from an order-1 Markov source over σ symbols: each symbol has 8 random successors, chosen with Zipf-like probabilities proportional to $j^{-1.5}$. The base sequence is copied 50 times, and each copy gets independent point mutations at rate μ , each replacing a symbol by a uniformly random symbol. We used $\sigma \in \{2, 4, 10, 100, 1000, 10^4, 10^5\}$ and $\mu \in \{0.1\%, 1\%\}$, plus a noise sweep at $\sigma = 10^5$. Uniformly random substitutions are harsh on the hybrid, because each one creates new bigrams and so new entries in the rank lists and new values in E . In real parses and word sequences, substitutions tend to be plausible symbols.

Method. For each setting we built eight indexes: the hybrid with each of the four structures on $\text{BWT}(E)$, and an index of T with each of the four structures on $\text{BWT}(T)$. All eight return identical counts on all queries, and on small inputs they match brute force. Queries are random substrings of T of length $m \in \{4, 8, 16, 32\}$, 20,000 per length (100,000 for $\sigma = 100000$ with 0.1% noise). We report $m = 32$; the other lengths are consistent. Because every query is a substring of T , every search runs to completion; with 50 copies of the base sequence, a pattern typically occurs about 50

σ	$H_0(T)$	σ_E	0.1% noise			1% noise		
			$r(T)$	$r(E)$	$r(\Psi_E)$	$r(T)$	$r(E)$	$r(\Psi_E)$
2	0.99	3	0.67	0.68	0.43	3.15	3.20	1.96
4	1.85	5	1.01	1.00	1.03	4.30	4.41	4.22
10	3.03	11	1.03	1.07	1.30	4.52	4.91	5.30
100	6.29	101	1.03	1.14	1.50	4.19	5.12	5.86
1000	9.66	264–920	1.00	1.15	1.53	3.79	5.14	6.01
10000	12.97	45–319	0.95	1.15	1.53	3.34	5.16	6.06
100000	16.22	18–55	0.93	1.15	1.59	2.90	5.37	6.61

Table 1: Numbers of runs, in millions, for $n = 5 \times 10^7$. σ_E is the alphabet size of E (the two values are for 0.1% and 1% noise where they differ). $r(T)$, $r(E)$ and $r(\Psi_E)$ are the runs in $\text{BWT}(T)$, $\text{BWT}(E)$ and Ψ_E .

times. Patterns that do not occur would usually stop early, which would favour the indexes with more expensive steps. The rows for $\sigma = 100000$ were measured in separate runs from the others. In them, the fast hybrid is about twice as fast as at $\sigma = 10000$, which we cannot explain and have not investigated; the difference may partly reflect load on the machine. The machine is a shared two-core virtual machine, and repeated runs vary by about $\pm 20\%$, so differences smaller than that should be read as ties. Sizes are exact.

Runs. Table 1 shows the number of runs. The $H_0(T)$ column gives the value for 0.1% noise. With 1% noise it is at most 0.01 bits higher: H_0 depends only on the symbol frequencies, which are set by the Markov source, and noise only moves a small fraction of the symbols toward a uniform distribution. Noise affects repetitiveness, which H_0 does not measure, and that shows up in the numbers of runs. $\text{BWT}(E)$ has up to 1.25 times as many runs as $\text{BWT}(T)$ with 0.1% noise, and up to 1.9 times as many with 1% noise. For $\sigma \geq 10$, Ψ_E has 1.3 to 2.3 times as many runs as $\text{BWT}(T)$. Together the two have 1.7 to 3.0 times as many runs as $\text{BWT}(T)$ at 0.1% noise, and 1.6 to 4.1 times as many at 1% noise. For $\sigma \leq 4$, Ψ_E has no more runs than $\text{BWT}(T)$, since there the encoding carries almost all the information. The extra runs are the main cost of the hybrid.

Space and time. Table 2 gives the query time and the size of each index. For each approach we show two variants: the fastest and the smallest.

- **Hybrid.** Fast: plain Huffman-shaped wavelet tree on $\text{BWT}(E)$. Compact: run-length Huffman-shaped wavelet tree on $\text{BWT}(E)$. Both use Elias–Fano run lists for Ψ_E and include the rank lists. Note that the fast hybrid is only partly run-length compressed: its wavelet tree takes about $n(H_0(E) + 1)$ bits, which grows with n rather than with r . Its size is moderate here only because $H_0(E)$ is about 2 bits. On a larger or more repetitive collection, it would fall further behind the run-length indexes in space.
- **FM-index of T .** Fast: plain Huffman-shaped wavelet tree. Compact: RLFM-index.
- **RLCSA of T .** Fast: explicit run lists. Compact: Elias–Fano run lists.

Observations.

- **Space.** The smallest index in every row is the Elias–Fano RLCSA of T , with the RLFM-index tied for small alphabets and up to 1.8 times larger for large ones. The compact hybrid is 1.7

noise	σ	Hybrid				FM-index of T				RLCSA of T			
		fast		compact		plain WT		RLFM		explicit		Elias-Fano	
0.1%	2	3.9	/ 9.6	7.5	/ 2.6	4.8	/ 9.7	9.9	/ 1.5	4.5	/ 5.3	7.3	/ 1.5
	4	8.7	/ 13.7	15.3	/ 4.5	10.2	/ 13.4	11.5	/ 2.3	8.9	/ 8.0	12.0	/ 2.3
	10	9.9	/ 15.6	10.8	/ 5.4	17.6	/ 20.7	16.2	/ 2.5	5.5	/ 8.2	13.0	/ 2.4
	100	11.8	/ 18.8	18.6	/ 6.7	34.6	/ 42.4	26.9	/ 2.9	7.4	/ 8.3	15.9	/ 2.8
	1000	11.5	/ 20.4	15.1	/ 8.1	56.0	/ 65.0	33.7	/ 3.3	11.1	/ 8.0	21.2	/ 3.2
	10000	13.1	/ 21.5	22.0	/ 9.3	67.4	/ 87.5	40.2	/ 4.0	11.1	/ 7.9	17.3	/ 3.5
	100000	6.9	/ 24.1	16.0	/ 13.7	104.5	/ 112.8	52.9	/ 8.1	2.7	/ 10.2	10.2	/ 4.4
1%	2	5.4	/ 12.3	15.9	/ 9.5	3.0	/ 9.7	19.2	/ 5.7	9.3	/ 25.5	13.2	/ 5.6
	4	7.8	/ 18.9	17.0	/ 14.9	8.5	/ 13.5	25.9	/ 7.5	8.1	/ 34.1	13.0	/ 7.5
	10	8.5	/ 22.6	28.6	/ 18.1	17.7	/ 20.7	36.3	/ 8.4	14.9	/ 36.2	20.2	/ 8.4
	100	12.2	/ 28.6	28.3	/ 22.3	29.9	/ 42.4	50.6	/ 9.8	15.9	/ 33.5	25.1	/ 9.4
	1000	14.4	/ 36.3	31.2	/ 29.9	47.6	/ 65.0	60.9	/ 10.7	16.1	/ 30.3	30.2	/ 10.2
	10000	13.9	/ 42.1	39.9	/ 35.8	77.9	/ 87.6	105.8	/ 11.6	20.3	/ 27.0	32.2	/ 10.5
	100000	7.2	/ 47.6	20.7	/ 43.3	86.5	/ 112.9	57.0	/ 15.2	4.6	/ 25.4	12.0	/ 11.1

Table 2: Counting time in μs per query for $m = 32$ / index size in MB, for $n = 5 \times 10^7$. In each row, the fastest query time is in bold red and the smallest size is in bold. The FM-index and RLCSA columns index T directly.

to 3.9 times larger than the Elias-Fano RLCSA, and the gap grows with σ and with noise. Most of the difference comes from Ψ_E and the rank lists. For example, at $\sigma = 10^5$ and 1% noise, the compact hybrid's 43.3 MB is 9.6 MB for BWT(E), 21.8 MB for Ψ_E and 11.9 MB for the rank lists.

- **Time, fast variants.** The fast hybrid's query time is nearly independent of σ , at 4 to 14 μs , because its wavelet tree has depth 1.3 to 2.5 bits per symbol for all σ . The plain FM-index of T slows down in proportion to $H_0(T)$, as the locality argument predicts. The explicit RLCSA slows down as the number of runs per character grows, and speeds up again at $\sigma = 10^5$, where each character has only about 10 runs.
- **Where the hybrid wins on time.** With 1% noise and $10 \leq \sigma \leq 10^4$, the fast hybrid is the fastest index, 1.1 to 1.8 times faster than the next one. At $\sigma = 1000$ the margin (14.4 against 16.1 μs) is within measurement noise, so the clear wins are for $\sigma = 10, 100$ and 10000. In those settings it is 2.7 to 4 times larger than the Elias-Fano RLCSA of T ; it is smaller than the explicit RLCSA for $\sigma = 10$ and 100, and larger for $\sigma = 1000$ and 10^4 . With 0.1% noise it is never clearly the fastest: it ties for $\sigma \leq 4, 1000$ and 10^4 , and the explicit RLCSA is clearly faster, and smaller, for $\sigma = 10, 100$ and 10^5 . For $\sigma \leq 4$ the encoding does nothing useful ($\sigma_E = \sigma + 1$).
- **Compact variants.** The compact hybrid is dominated on this data: it is always larger than the Elias-Fano RLCSA of T , and about as fast or up to 1.7 times slower.
- **More noise at $\sigma = 10^5$.** At noise levels of 0.5%, 2% and 5%, the fast hybrid takes 35, 70 and 125 MB and 6.8, 10.3 and 15.4 μs , against 17, 42 and 89 MB and 3.1, 7.3 and 13.1 μs for the explicit RLCSA of T . The RLCSA degrades faster with noise, but it stays ahead.

In short, on this data the fast hybrid offers a different point on the space-time curve: at intermediate alphabet sizes with some noise it is the fastest index, at 2.7 to 4 times the space of the

smallest one. It is never the smallest, and when space matters most, the compressed RLCSA of T is the better choice.

7 Discussion and open questions

Runs on real data. The hybrid’s size is dominated by the runs of $\text{BWT}(E)$ and especially Ψ_E . Our generator replaces symbols with uniformly random ones, which creates many new bigrams: at $\sigma = 10^5$, going from 0.1% to 5% noise increases the number of distinct bigrams from 0.47 to 5.1 million. Real parses, minimizer digests and word sequences should behave better. Measuring $r(E)$ and $r(\Psi_E)$ on real data is the first thing to do. Bounds on $r(\Psi_E)$ in terms of $r(T)$ or other repetitiveness measures would also be interesting. Run boundaries in Ψ_E correspond to distinct substrings of T , which suggests a δ -style bound.

Smaller hybrids. In the compact hybrid, the rank lists take about a quarter of the space and are stored naively. They could be compressed, for example with Elias–Fano for the successor identifiers. An order- k version would rank each character by the k characters before it and store k -grams in Ψ_E . That makes E more skewed and Ψ_E bigger, and it is not clear which effect wins.

Move structures. Move structures [18], as used in Movi [24], represent a BWT interval by the runs containing its ends. To extend the interval by c , when the run at an end is not a run of c , they scan to the nearest run of c . Over a large alphabet those scans can be long. The standard fixes store, at each run or at sampled runs, pointers or counts for every character, which costs space proportional to σ per sample. A rough model shows the problem does not go away with skew. Suppose the runs of c are spread evenly, about r/r_c runs apart, and suppose the pattern’s characters appear in proportion to their numbers of runs. Then the expected scan length is $\sum_c (r_c/r)(r/r_c) = \sigma$ runs, whatever the distribution. This is why we have argued that move structures are best suited to small alphabets.

The hybrid suggests a combination: a move structure on $\text{BWT}(E)$, followed by one CSA-style step on Ψ_E , which is a binary search and needs no scanning. The rough model says the scans then cost about σ_E runs. That is small on clean data (18 at $\sigma = 10^5$) but grows with noise (210 at 5% noise).

Other open questions.

- Implementing and evaluating locating, including the ϕ -based approach of Section 3.4.
- Comparing against the relabeled-block FM-index baseline of Section 4, and against a tuned RLFM-index.
- Using the hybrid for the parse level of a two-level index [3], which is where large, repetitive alphabets first motivated this work.

Acknowledgments

The idea of the hybrid is due to the author. Claude (Anthropic) wrote the prototype and ran the experiments, worked out the locating sketch in Section 3.4 and the move-structure estimate in Section 7, and wrote this document, in conversation with the author.

References

- [1] O. Y. Ahmed, M. Rossi, T. Gagie, C. Boucher, and B. Langmead. SPUMONI 2: improved classification using a pangenome index of minimizer digests. *Genome Biology*, 24:122, 2023.
- [2] J. W. Bentley, D. Gibney, and S. V. Thankachan. On the complexity of BWT-runs minimization via alphabet reordering. In *Proc. 28th European Symposium on Algorithms (ESA)*, 2020.
- [3] N. K. Brown, T. Gagie, G. Manzini, G. Navarro, and M. Sciortino. Faster run-length compressed suffix arrays. In *From Strings to Graphs, and Back Again: A Festschrift for Roberto Grossi’s 60th Birthday*, OASICs, article 10, 2025.
- [4] B. Chapin and S. R. Tate. Higher compression from the Burrows–Wheeler transform by modified sorting. In *Proc. Data Compression Conference (DCC)*, 1998.
- [5] J.-J. Deng, W.-K. Hon, D. Köppl, and K. Sadakane. FM-indexing grammars induced by suffix sorting for long patterns. In *Proc. Data Compression Conference (DCC)*, 2022.
- [6] A. Fariña, N. R. Brisaboa, G. Navarro, F. Claude, Á. S. Places, and E. Rodríguez. Word-based self-indexes for natural language text. *ACM Transactions on Information Systems*, 30(1):1, 2012.
- [7] P. Fenwick. Symbol ranking text compression with Shannon recodings. *Journal of Universal Computer Science*, 3(2):70–85, 1997.
- [8] P. Ferragina and G. Manzini. Indexing compressed text. *Journal of the ACM*, 52:552–581, 2005.
- [9] T. Gagie, G. Navarro, and N. Prezza. Fully functional suffix trees and optimal text searching in BWT-runs bounded space. *Journal of the ACM*, 67:1–54, 2020.
- [10] R. Giancarlo, G. Manzini, A. Restivo, G. Rosone, and M. Sciortino. A new class of string transformations for compressed text indexing. *Information and Computation*, 294:105068, 2023.
- [11] I. Grebnov. libsais: fast linear-time suffix array, LCP and BWT construction. <https://github.com/IlyaGrebnov/libsais>.
- [12] R. Grossi, A. Gupta, and J. S. Vitter. High-order entropy-compressed text indexes. In *Proc. 14th ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2003.
- [13] R. Grossi and J. S. Vitter. Compressed suffix arrays and suffix trees with applications to text indexing and string matching. *SIAM Journal on Computing*, 35:378–407, 2005.
- [14] A. Hong, M. Oliva, D. Köppl, H. Bannai, C. Boucher, and T. Gagie. PFP-FM: an accelerated FM-index. *Algorithms for Molecular Biology*, 19:15, 2024.
- [15] D. Kempa and T. Kociumaka. Resolution of the Burrows–Wheeler transform conjecture. *Communications of the ACM*, 65(6):91–98, 2022.
- [16] V. Mäkinen and G. Navarro. Succinct suffix arrays based on run-length encoding. *Nordic Journal of Computing*, 12:40–66, 2005.
- [17] V. Mäkinen, G. Navarro, J. Sirén, and N. Välimäki. Storage and retrieval of highly repetitive sequence collections. *Journal of Computational Biology*, 17:281–308, 2010.
- [18] T. Nishimoto and Y. Tabei. Optimal-time queries on BWT-runs compressed indexes. In *Proc. 48th International Colloquium on Automata, Languages, and Programming (ICALP)*, 2021.
- [19] D. Okanohara and K. Sadakane. Practical entropy-compressed rank/select dictionary. In *Proc. 9th Workshop on Algorithm Engineering and Experiments (ALENEX)*, 2007.

- [20] A. Ordóñez, G. Navarro, and N. R. Brisaboa. Grammar compressed sequences with rank/select support. *Journal of Discrete Algorithms*, 43:54–71, 2017.
- [21] K. Sadakane. New text indexing functionalities of the compressed suffix arrays. *Journal of Algorithms*, 48:294–313, 2003.
- [22] J. Sirén, E. Garrison, A. M. Novak, B. Paten, and R. Durbin. Haplotype-aware graph indexes. *Bioinformatics*, 36(2):400–407, 2020.
- [23] J. Sirén. *Compressed Full-Text Indexes for Highly Repetitive Collections*. PhD thesis, University of Helsinki, 2012.
- [24] M. Zakeri, N. K. Brown, O. Y. Ahmed, T. Gaggie, and B. Langmead. Movi: a fast and cache-efficient full-text pangenome index. *iScience*, 27, 2024.